Projekat 1 Šel programiranje

DataPipe: robusni ETL pipeline sa supervizorom

Oktobar 2026.

1 Opšti zahtev

Potrebno je razviti bash skriptu `pipeline.sh` koja upravlja obradom velike količine podataka. Samu obradu podataka ne radite vi: ona je već implementirana u Python programu `process.py`, koji dobijate gotov. Vaš zadatak je da napišete **infrastrukturu oko tog programa**: da podelite veliki ulazni fajl na delove, da delove obrađujete paralelno, da nadgledate pokrenute procese, da ponovite neuspele obrade, da prepoznate i ubijete procese koji su se zaglavili, da omogućite nastavak prekinutog rada i da na kraju spojite i proverite rezultate.

Program `process.py` **namerno nije pouzdan**: ponekad pukne, ponekad se zaglavi, ponekad ga ubije sistem, a ponekad završi "uspešno" ali ostavi neispravan izlaz. Dobra skripta mora da preživi sve te situacije i da na kraju da tačan rezultat.

Ovaj projekat treba da proveri razumevanje gradiva koje ste prošli u prethodnoj polovini kursa: procesi, signali, izlazni kodovi, paralelno izvršavanje, zaključavanje, obrada teksta (`awk`, `sed`, `sort`) i rad sa fajlovima.

Projekat se radi **samostalno** i vredi do **35 poena**.

2 Logistički zahtevi

- Projekat predajete preko github classroom sistema, gde ćete dobiti privatni repozitorijum za vaš rad.
- Repozitorijum će da bude u formatu ime_prezime_indeks (SI_<III broj indeksa 3 cifre>_<GGGG godina 4 cifre>)

- Početni repozitorijum sadrži folder `provided/` sa programima `process.py` i `gen_data.py`. Ove fajlove **ne smete menjati**. Prilikom ocenjivanja koristi se originalna kopija ovih programa.
- U repozitorijumu treba da se nalazi kod u fajlu `pipeline.sh` i makar jedan konfig fajl `pipeline.conf`. Dozvoljeno je podeliti kod na više fajlova u folderu `lib/`, koje `pipeline.sh` učitava.
- Velike ulazne fajlove i folder `runs/` **ne stavljate** u repozitorijum (dodajte ih u `.gitignore`).
- Rok za predaju projekta je **13. Novembar 2026.** u 23:59h.

Dozvoljeni alati

Skripta mora da radi u **bash**-u verzije 4.4 ili novije. Dozvoljeno je korišćenje standardnih alata: coreutils (`split`, `sort`, `wc`, `df`, `stat`, `timeout`, ...), `awk`, `sed`, `grep`, `find`, `flock`, `ps`, `kill`, `xargs`. **Nije dozvoljeno**: pisanje delova rešenja u Python-u ili drugom jeziku (osim pozivanja `process.py`), korišćenje alata GNU `parallel`, kao i učitavanje konfig fajla pomoću `source`, `.` ili `eval`.

3 Opis sistema

Tok obrade jednog pokretanja (*run*) je sledeći:

```
data.csv  --(deljenje)-->  chunk_00001.csv ... chunk_000NN.csv
                                        |
                      (N paralelnih radnika, nadzor, retry)
                                        |
                                process.py  (za svaki deo)
                                        |
           results/chunk_00001.out ... results/chunk_000NN.out
                                        |
                        (validacija + spajanje)
                                        |
                                  result.csv
```

4 Dati programi

Generator podataka: `gen_data.py`

Služi za pravljenje test podataka proizvoljne veličine. Pored podataka može da napravi i fajl sa tačnim očekivanim rezultatom, sa kojim možete da uporedite svoj `result.csv`.

```
python3 provided/gen_data.py --rows 2000000 --seed 42 --output data.
    csv \
        [--bad-rate 0.0005] [--group-by region] [--expected expected
    .csv]
```

Ulazni CSV fajl ima zaglavlje i sledeće kolone (separator je zarez, polja nisu pod navodnicima):

```
transaction_id,timestamp,region,category,amount,quantity
1,2026-03-14T10:22:31,Novi Sad,elektronika,1299.99,1
2,2026-03-14T10:22:35,Niš,hrana,349.50,3
```

Mali procenat redova (`-bad-rate`) je namerno neispravan (fali polje, iznos nije broj, ...). Obratite pažnju da vrednosti mogu da sadrže razmake i karaktere van ASCII skupa (`Novi Sad`, `Niš`).

Program za obradu: `process.py`

Ovo je **ugovor** (interfejs) programa. Vaša skripta sme da se oslanja samo na ono što je ovde opisano.

Pokretanje

```
python3 provided/process.py --input <chunk.csv> --output <rezultat>
    [--group-by region|category]
```

Ulaz je jedan deo podataka sa zaglavljem (isti format kao `data.csv`). Podrazumevano grupisanje je po koloni `region`.

Izlaz Program računa agregate po grupi. Izlazni fajl ima zaglavlje, jedan red po grupi (**redosled grupa nije definisan**) i završnu kontrolnu liniju `#END`:

```
group,count,sum_amount,min_amount,max_amount,sum_quantity
Beograd,24987,12493512.33,0.99,4999.99,74890
Novi Sad,12011,6012332.10,1.20,4999.50,36120
...
#END rows_in=100000 rows_skipped=4 groups=6
```

- `rows_in` je broj pročitanih redova podataka (bez zaglavlja), uključujući i preskočene.
- `rows_skipped` je broj neispravnih redova koji nisu ušli u agregate.
- `groups` je broj redova sa grupama u fajlu.

Izlaz je **ispravan** samo ako postoji tačno zaglavlje, ako postoji linija `#END`, ako je `rows_in` jednak broju redova podataka u ulaznom delu i ako je `groups` jednak stvarnom broju redova sa grupama. Ako program ne uspe, može da ostavi **nepotpun** izlazni fajl.

Standardni izlaz za greške (stderr) Program ispisuje svoj napredak na stderr, jedna poruka po liniji:

```
START input=chunk_00003.csv pid=41872
PROGRESS rows=5000
PROGRESS rows=10000
WARN skipped_row line=7731 reason=bad_amount
...
DONE rows=100000 skipped=4 elapsed=2.41
```

U slučaju greške poslednja linija je `ERROR <poruka>`. **Garancija**: dok radi ispravno, program ispiše novu liniju na stderr **najmanje jednom u sekundi**. Ako duže vreme nema novih linija, program se zaglavio.

Izlazni kodovi

Kod	Značenje
0	Uspeh. (Pažnja: i pored koda 0, izlaz može biti neispravan i mora se proveriti.)
1	Privremena greška. Ponovni pokušaj ima smisla.
2	Greška u pozivu ili formatu ulaza (loši argumenti, ulaz ne postoji, pogrešno zaglavlje). Ponovni pokušaj nema smisla.
143	Proces je završen signalom `SIGTERM` (128+15).
137	Proces je ubijen signalom `SIGKILL` (128+9), od strane vas ili sistema.

Signali Na `SIGTERM` program briše svoj nepotpun izlaz i završava se u roku od jedne sekunde sa kodom 143. Postoje situacije zaglavljivanja u kojima program **ignoriše** `SIGTERM`; tada ga jedino `SIGKILL` zaustavlja.

Ubacivanje grešaka (za testiranje) Ponašanje programa se kontroliše promenljivama okruženja. Koristite ih da testirate svoju skriptu. Prilikom ocenjivanja koristiće se **skriveni** plan grešaka.

Promenljiva	Značenje
`PROC_FAIL_RATE`	verovatnoća (0–1) da program pukne sa kodom 1
`PROC_HANG_RATE`	verovatnoća da se program zaglavi (reaguje na `SIGTERM`)
`PROC_HARDHANG_RATE`	verovatnoća da se zaglavi i ignoriše `SIGTERM`
`PROC_CRASH_RATE`	verovatnoća da program ubije sam sebe sa `SIGKILL`
`PROC_CORRUPT_RATE`	verovatnoća da vrati 0 a ostavi nepotpun izlaz
`PROC_SLOW_MS`	dodatno kašnjenje u ms na svakih 1000 redova
`PROC_SEED`	seme za ponovljivo ponašanje
`PROC_FAULT_PLAN`	fajl sa tačnim planom grešaka (vidi ispod)

Fajl plana grešaka ima po jednu liniju oblika `<ime dela> <broj pokušaja> <greška>`, gde je greška jedna od `fail`, `hang`, `hardhang`, `crash`, `corrupt`:

```
# drugi deo pukne u prvom pokušaju, a uspe u drugom
chunk_00002.csv 1 fail
# peti deo se zaglavi u prva dva pokušaja
```

```
chunk_00005.csv 1 hang
chunk_00005.csv 2 hardhang
```

Program sam broji pokušaje po imenu ulaznog fajla u direktorijumu `PROC_STATE_DIR` (podrazumevano `${TMPDIR:-/tmp}/process_py_state`). Obrišite taj direktorijum pre svakog testa.

5 Zahtevi

Skripta sadrži nekoliko režima rada, koje korisnik bira pri pokretanju:

- `./pipeline.sh run -input <fajl> [-config <fajl>] [-workers N]` – pokreće novu obradu.
- `./pipeline.sh resume <run_id>` – nastavlja prekinutu obradu.
- `./pipeline.sh status [<run_id>]` – prikazuje stanje obrade (podrazumevano poslednje).
- `./pipeline.sh report <run_id>` – ispisuje izveštaj o završenoj obradi.
- `./pipeline.sh list` – lista sva pokretanja.
- `./pipeline.sh clean -keep N` – briše stara pokretanja, čuva N najnovijih.
- `./pipeline.sh -help` – ispisuje uputstvo za korišćenje.

Ceo projekat se sastoji od nekoliko funkcionalnih celina koje treba da implementirate.

Konfiguracija i CLI

Konfig fajl (podrazumevano `pipeline.conf` pored skripte) ima format `ključ=vrednost`. Pravila:

- Linije koje počinju sa `#` i prazne linije se ignorišu.
- Razmaci oko `=` su dozvoljeni i uklanjaju se.

- Nepoznat ključ se loguje kao `WARN` i ignoriše.
- Neispravna vrednost (npr. tekst gde se očekuje broj) je greška: skripta se završava sa kodom 2.
- Prioritet vrednosti: opcija komandne linije > konfig fajl > podrazumevana vrednost.

```
# pipeline.conf
processor = ./provided/process.py
python = python3
group_by = region
chunk_lines = 100000
workers = 4
max_retries = 3
backoff_base = 1
backoff_max = 30
hang_timeout = 10
kill_grace = 3
max_failed_chunks = 2
work_dir = ./runs
keep_chunks = no
log_level = INFO
```

Ključ	Obavezan	Podrazum.	Značenje
`processor`	da	–	putanja do `process.py`
`python`	ne	`python3`	interpreter kojim se pokreće processor
`group_by`	ne	`region`	`region` ili `category`, prosleđuje se kao `-group-by`
`chunk_lines`	ne	100000	broj redova podataka po delu (pozitivan ceo broj)
`workers`	ne	4	najveći broj istovremenih `process.py` procesa
`max_retries`	ne	3	broj ponovnih pokušaja po delu (ukupno pokušaja: 1 + `max_retries`)

Ključ	Obavezan	Podrazum.	Značenje
`backoff_base`	ne	1	osnova čekanja pre ponovnog pokušaja, u sekundama
`backoff_max`	ne	30	najduže čekanje pre ponovnog pokušaja, u sekundama
`hang_timeout`	ne	10	broj sekundi bez novog reda na stderr posle kojeg se proces smatra zaglavljenim
`kill_grace`	ne	3	sekunde između `SIGTERM` i `SIGKILL`
`max_failed_chunks`	ne	2	ako broj trajno neuspelih delova pređe ovu vrednost, obrada se prekida
`work_dir`	ne	`./runs`	gde se čuvaju pokretanja
`keep_chunks`	ne	`no`	`yes`/`no`: da li se delovi čuvaju posle uspešne obrade
`log_level`	ne	`INFO`	`DEBUG`, `INFO`, `WARN` ili `ERROR`

Struktura radnog direktorijuma

Svako pokretanje dobija identifikator oblika `run_YYYYmmdd_HHMMSS` i svoj direktorijum. Struktura je **obavezna**, jer je koristi test skripta:

```
runs/
  history.csv                    # istorija svih pokretanja
  run_20261020_101500/
    run.meta                     # kljuc=vrednost: input, started,
    finished, status, ...
    config.snapshot              # konfiguracija sa kojom je
    pokretanje zapoceto
    manifest.csv                 # stanje svakog dela
    pipeline.log                 # log skripte
    chunks/chunk_00001.csv ...   # delovi ulaza (svaki sa zaglavljem)
    results/chunk_00001.out ... # ISKLJUCIVO ispravni rezultati
    logs/chunk_00001.1.err ...   # stderr svakog pokusaja (deo.
    pokusaj.err)
    result.csv                   # konacni rezultat
```

```
report.txt                  # izvestaj
```

Status pokretanja (`status` u `run.meta`) je jedan od: `running`, `completed`, `partial`, `aborted`, `interrupted`, `failed`.

Deljenje ulaza

Ulazni fajl se deli na delove od po `chunk_lines` redova podataka. Svaki deo mora da počinje istim zaglavljem kao ulaz. Delovi se imenuju `chunk_00001.csv`, `chunk_00002.csv`, ... (pet cifara). Poslednji deo može biti manji. Pre deljenja proverite da zaglavlje ulaza odgovara formatu iz specifikacije. Posle deljenja proverite da je zbir redova podataka u svim delovima jednak broju redova podataka u ulazu.

Paralelna obrada

Delovi se obrađuju paralelno, tako da u svakom trenutku radi **najviše** `workers` procesa `process.py` (ovo se proverava za vreme testa). Čim jedan proces završi, sledeći deo treba da krene. Stanje delova se vodi u `manifest.csv`, jedan red po delu:

```
chunk,status,attempts,rows,duration_s,exit_code,reason
chunk_00001,done,1,100000,2.41,0,
chunk_00002,done,2,100000,2.87,0,error
chunk_00003,failed,4,100000,0.00,137,hang
chunk_00004,running,1,100000,,,
chunk_00005,pending,0,100000,,,
```

- `status`: `pending`, `running`, `done` ili `failed`.
- `reason`: razlog poslednjeg neuspelog pokušaja: `error`, `usage_error`, `hang`, `crash`, `corrupt`, `interrupted` ili prazno.

Više radnika menja isti fajl istovremeno, pa izmene `manifest.csv` moraju da budu zaštićene zaključavanjem (npr. `flock`). Nijedna izmena ne sme da se izgubi.

Ponovni pokušaji i klasifikacija grešaka

Posle svakog pokušaja skripta odlučuje šta dalje, na osnovu izlaznog koda i provere izlaza:

Situacija	Postupak
kod 0 i izlaz ispravan	deo je `done`, rezultat se **atomično** premešta u `results/`
kod 0 a izlaz neispravan	razlog `corrupt`, ponoviti
kod 1	razlog `error`, ponoviti
kod 137 (ne od vas)	razlog `crash`, ponoviti
zaglavljivanje	razlog `hang`, ponoviti
kod 2	razlog `usage_error`, **ne ponavljati**, deo odmah postaje `failed`
nepoznat kod	zapisati `WARN` i ponoviti

Pre k-tog ponovnog pokušaja čeka se $\min(\texttt{backoff_base} \cdot 2^{k-1}, \texttt{backoff_max})$ sekundi. Deo koji ne uspe ni posle $1 + \texttt{max_retries}$ pokušaja postaje `failed`. Nepotpun izlaz neuspelog pokušaja nikada ne sme da završi u `results/`.

Ako broj delova u stanju `failed` pređe `max_failed_chunks`, obrada se **prekida**: novi delovi se ne pokreću, pokrenuti procesi se zaustavljaju, status pokretanja postaje `aborted`, a skripta se završava sa kodom 4.

Detekcija zaglavljivanja

Za svaki pokrenut `process.py` skripta prati njegov stderr fajl (`logs/<deo>.<pokušaj>.err`). Ako se za `hang_timeout` sekundi ne pojavi nijedna nova linija, proces je zaglavljen:

1. zapisati `WARN` u log,
2. poslati `SIGTERM` **tom** `process.py` procesu (ne celoj grupi i ne sebi),
3. ako proces i dalje radi posle `kill_grace` sekundi, poslati `SIGKILL`,
4. zabeležiti razlog `hang` i postupiti kao kod ostalih grešaka koje se ponavljaju.

Napomena: samo komanda `timeout` **nije dovoljna**, jer ona meri ukupno trajanje procesa, a ne vreme bez napretka. Proces koji radi dugo ali redovno javlja napredak **ne sme** biti ubijen.

Prekid i nastavak rada

- Kada `pipeline.sh` primi `SIGINT` (Ctrl+C) ili `SIGTERM`, treba da prestane da pokreće nove delove, da zaustavi sve pokrenute `process.py` procese (`SIGTERM`, pa `SIGKILL` posle `kill_grace`), da vrati delove koji su bili `running` u `pending` (razlog `interrupted`), da postavi status `interrupted` i da se završi sa kodom 130. Posle prekida **ne sme ostati nijedan** `process.py` proces.
- `./pipeline.sh resume <run_id>` nastavlja obradu sa konfiguracijom iz `config.snapshot`. Delovi koji su `done` se ne obrađuju ponovo, ali se njihov rezultat prvo proveri (ako je neispravan ili ne postoji, deo se vraća u `pending`). Delovi koji su ostali `running` (npr. posle `kill -9` same skripte) vraćaju se u `pending`. Delovi koji su `failed` ostaju `failed`.
- Nad jednim pokretanjem u isto vreme sme da radi samo jedna instanca skripte. Druga instanca (`resume` istog `run_id`) se završava odmah, sa porukom o grešci i kodom 3.

Spajanje i validacija rezultata

Kada nema više delova u stanju `pending` ili `running`, rezultati iz `results/` se spajaju u jedan fajl. Pre spajanja svaki rezultat se ponovo proverava (pravila iz opisa izlaza `process.py`). Konačni rezultat ima format:

```
group,count,sum_amount,avg_amount,min_amount,max_amount,sum_quantity
Beograd,499870,249893511.07,499.92,0.99,4999.99,1499102
...
```

- `count`, `sum_amount` i `sum_quantity` se sabiraju, `min_amount` je minimum minimuma, `max_amount` je maksimum maksimuma.
- `avg_amount` = `sum_amount`/`count`.
- Svi iznosi se ispisuju sa tačno dve decimale.
- Redovi su sortirani po koloni `group` sa `LC_ALL=C sort`.

- Fajl se prvo piše u privremeni fajl, pa se premešta na konačno mesto (nikada ne sme postojati poluupisan `result.csv`).

Ako su svi delovi `done`, rezultat se upisuje u `result.csv`, status je `completed`, a izlazni kod 0. Ako postoje `failed` delovi (ali ne više od `max_failed_chunks`), rezultat se upisuje u `result.partial.csv`, status je `partial`, a izlazni kod 1. Ako je `keep_chunks=no` i status je `completed`, folder `chunks/` se briše.

Vaš `result.csv` mora da se poklapa sa fajlom koji pravi `gen_data.py -expected` (za isto seme, isti broj redova i isto grupisanje). Jedino kolona `avg_amount` sme da odstupa za 0.01 u retkim slučajevima zaokruživanja.

Evidentiranje rada skripte

Log Svaka akcija se upisuje u `pipeline.log` u formatu `[YYYY-MM-DD HH:MM:SS] NIVO: poruka`. Poruke ispod `log_level` se ne upisuju. Poruke nivoa `ERROR` se ispisuju i na stderr.

```
[2026-10-20 10:15:00] INFO: Run run_20261020_101500 started (input=
   data.csv, workers=4)
[2026-10-20 10:15:02] INFO: Split into 20 chunks (2000000 rows)
[2026-10-20 10:15:02] INFO: chunk_00001 attempt 1 started (pid
   41872)
[2026-10-20 10:15:05] WARN: chunk_00002 attempt 1 failed (exit=1,
   reason=error), retry in 1s
[2026-10-20 10:15:19] WARN: chunk_00005 attempt 1 hang detected (no
   progress for 10s), sending SIGTERM
[2026-10-20 10:15:22] WARN: chunk_00005 still alive after 3s,
   sending SIGKILL
[2026-10-20 10:16:40] INFO: Merged 20 results into result.csv
[2026-10-20 10:16:40] INFO: Run finished: status=completed, duration
   =100s
```

Istorija Na kraju svakog pokretanja (uspešnog ili ne) dodaje se red u `runs/history.csv`:

```
run_id,started,finished,status,chunks,failed,rows,duration_s
run_20261020_101500,2026-10-20 10:15:00,2026-10-20 10:16:40,
   completed,20,0,2000000,100
```

Izveštaj Na kraju pokretanja pravi se `report.txt`, a `./pipeline.sh report <run_id>` ga ispisuje:

```
Run:             run_20261020_101500
Input:           data.csv (2000000 rows, 98.4 MB)
Status:          completed
Chunks:          20 total, 20 done, 0 failed
Attempts:        26 (6 retries)
Failures:        error=3 crash=1 corrupt=1 hang=1 usage_error=0
Hang kills:      1 SIGTERM, 1 SIGKILL
Duration:        100s
Throughput:      20000 rows/s
Slowest chunks:  chunk_00005 (14.20s), chunk_00011 (3.10s),
    chunk_00002 (2.87s)
Failed chunks:   -
```

Status, list i clean

- `status` prikazuje status pokretanja, broj delova po stanju, i za svaki deo koji je `running`: broj pokušaja, koliko dugo radi i poslednji `PROGRESS` red. Mora da radi i dok je obrada u toku (iz drugog terminala).
- `list` prikazuje sva pokretanja: `run_id`, status, vreme početka, broj delova i trajanje.
- `clean -keep N` briše sva pokretanja osim N najnovijih. Pokretanje koje je trenutno zaključano (radi) se ne briše.

Obrada grešaka

Skripta treba da proveri uslove pre početka rada i da korisnika jasno obavesti o problemu (poruka na stderr, `ERROR` u logu, izlazni kod 2):

- ne postoji `python` interpreter ili `processor` fajl,
- ulazni fajl ne postoji, nije čitljiv, prazan je ili ima pogrešno zaglavlje,

- `work_dir` ne može da se napravi ili u njega ne može da se piše,
- na disku nema bar dvostruko više slobodnog prostora od veličine ulaza (delovi + rezultati),
- konfig fajl ne postoji ili sadrži neispravnu vrednost, nepoznat režim rada ili nepoznata opcija.

Izlazni kodovi skripte

Kod	Značenje
0	svi delovi uspešno obrađeni, napravljen `result.csv`
1	obrada završena, ali postoje `failed` delovi (napravljen `result.partial.csv`)
2	greška u pozivu, konfiguraciji ili ulazu
3	nad pokretanjem već radi druga instanca
4	obrada prekinuta jer je pređen `max_failed_chunks`
130	obrada prekinuta signalom (`SIGINT`/`SIGTERM`)

6 Primer sesije

```
$ python3 provided/gen_data.py --rows 2000000 --seed 42 --output
    data.csv --expected expected.csv
$ rm -rf /tmp/process_py_state
$ export PROC_FAIL_RATE=0.1 PROC_HANG_RATE=0.05 PROC_SEED=7
$ ./pipeline.sh run --input data.csv --workers 4
Run run_20261020_101500 started (20 chunks, 4 workers)
...
Run finished: completed (result: runs/run_20261020_101500/result.csv
    )
$ echo $?
0
$ diff runs/run_20261020_101500/result.csv expected.csv && echo OK
OK

$ ./pipeline.sh run --input data.csv        # Ctrl+C posle nekoliko
    sekundi
^C Interrupted, stopping 4 running chunks...
```

```
$ echo $?
130
$ pgrep -f process.py || echo "nema zaostalih procesa"
nema zaostalih procesa
$ ./pipeline.sh resume run_20261020_102000
Resuming run_20261020_102000: 7 done, 13 pending
...
```

7 Testiranje

Projekat će biti testiran automatskom test skriptom koja koristi skriveni plan grešaka (`PROC_FAULT_PLAN`), šalje signale vašoj skripti i za vreme rada proverava broj pokrenutih procesa. Zato se **formati fajlova, imena, struktura direktorijuma i izlazni kodovi moraju poštovati tačno** kako su navedeni u ovom dokumentu.

8 Bodovanje

Maksimalan broj poena je 35, a raspodela je sledeća:

- Konfiguracija i CLI - 3 poena
- Deljenje ulaza - 3 poena
- Paralelna obrada (sa `manifest.csv` i zaključavanjem) - 4 poena
- Ponovni pokušaji i klasifikacija grešaka - 4 poena
- Detekcija zaglavljivanja - 4 poena
- Prekid i nastavak rada (signali, `resume`, lock) - 4 poena
- Spajanje i validacija rezultata - 4 poena
- Evidentiranje rada skripte (log, istorija, izveštaj, `status`, `list`, `clean`) - 2 poena
- Obrada grešaka - 2 poena

- Preostalih 5 poena se dodeljuju na odbrani projekta, na osnovu kratke izmene projekta, koju radite na licu mesta

9 Odbrana

Na odbrani ćete dobiti jednu manju izmenu (5–10 minuta) koju treba da uradite u **svom** kodu, da pokažete da ga razumete. Izmena može biti nova funkcionalnost u vašoj skripti, ili **nova verzija programa** `process.py` sa izmenjenim ugovorom, na koju vaša skripta treba da se prilagodi. Pre odbrane proverite da vaš projekat radi na čistom okruženju.

Zavisno od koliko ste dobro odbranili projekat, moze da dođe do korekcije bodova iz prethodnih delova, na sledeći način:

Nivo odbrane	**Koeficijent**
Super odbrana	1.0
Dobra odbrana	0.8
Solidna odbrana	0.6
Nedovoljna odbrana	0.4
Neuspešna odbrana	0.2